Shell Scripting Practice - Solutions

Solutions to the six practice assignments, with tested scripts and sample output

Bash scripts using read, case, if, find and bc

Index

***.** How to create and run a shell script (applies to all questions)

1. Write a shell script program to find area of a Square, Rectangle, Triangle and Circle, accept the inputs from the user.

2. Create a shell script that finds and lists all empty files in a /root directory.

3. Write a script to find whether entered number is even or odd.

4. Write a program using case statement to perform Character Type Checker (Vowel, Consonant, Digit, Special Character).

5. Write a program using case statement to perform Linux Command Information (menu: pwd, date, whoami, hostname).

6. Write a program using case statement for Linux User Management (Create, Delete, Lock, Unlock, List Users) - display the corresponding commands.

Item marked * is a general guide to creating and running scripts.

How to create and run a shell script (applies to all questions)

1. Create the file with an editor: `vi script.sh` (or `nano script.sh`) and type the script.
2. The first line `#!/bin/bash` (the **shebang**) tells the system to run the script with the bash shell.
3. Give execute permission: `chmod +x script.sh`.
4. Run it: `./script.sh` (or without execute permission: `bash script.sh`).
5. Scripts that use `useradd`, `userdel` or read `/root` must be run as root (or with `sudo`).

Element used	Meaning
read -p "text" var	Displays a prompt and stores the user's input in variable var
$var	Value of a variable
$(command)	Command substitution: output of the command is used as a value
bc -l	Calculator that supports decimal (floating-point) arithmetic; plain bash handles integers only
case ... esac	Multi-way selection; each pattern ends with `;;`, `*)` is the default
if / elif / else / fi	Conditional statement
$(( ))	Integer arithmetic expansion
[[$x =~ regex]]	Regular-expression match test

Note: `bc` may need installing (`yum install bc` or `apt install bc`). All scripts below were tested before being included; the sample runs show the user's input after each prompt.

Q1. Write a shell script program to find area of a Square, Rectangle, Triangle and Circle, accept the inputs from the user.

Formulas

Shape	Formula	Inputs
Square	side x side	side
Rectangle	length x breadth	length, breadth
Triangle	1/2 x base x height	base, height
Circle	3.14159 x r x r (pi x r squared)	radius

Script: q1_area.sh

```
#!/bin/bash
# Q1: Area of Square, Rectangle, Triangle and Circle

echo "===== AREA CALCULATOR ====="
echo "1. Square"
echo "2. Rectangle"
echo "3. Triangle"
echo "4. Circle"
read -p "Enter your choice (1-4): " choice

case $choice in
  1)
    read -p "Enter side of square: " side
    area=$(echo "$side * $side" | bc -l)
    echo "Area of Square = $area"
    ;;
  2)
```

```
    read -p "Enter length: " l
    read -p "Enter breadth: " b
    area=$(echo "$l * $b" | bc -l)
    echo "Area of Rectangle = $area"
    ;;
  3)
    read -p "Enter base: " base
    read -p "Enter height: " height
    area=$(echo "0.5 * $base * $height" | bc -l)
    echo "Area of Triangle = $area"
    ;;
  4)
    read -p "Enter radius: " r
    area=$(echo "3.14159 * $r * $r" | bc -l)
    printf "Area of Circle = %.2f\n" "$area"
    ;;
  *)
    echo "Invalid choice! Please select 1 to 4."
    ;;
esac
```

Sample output

```
$ ./q1_area.sh
===== AREA CALCULATOR =====
1. Square
2. Rectangle
3. Triangle
4. Circle
Enter your choice (1-4): 1
Enter side of square: 5
Area of Square = 25

Enter your choice (1-4): 2      (length 6, breadth 4)  ->  Area of Rectangle = 24
Enter your choice (1-4): 3      (base 10, height 5)    ->  Area of Triangle = 25.0
Enter your choice (1-4): 4      (radius 7)             ->  Area of Circle = 153.94
Enter your choice (1-4): 9      ->  Invalid choice! Please select 1 to 4.
```

Explanation

- A menu lets the user select the shape; a `case` statement runs the matching block.
- Bash cannot multiply decimals, so the expression is piped to `bc -l`, which performs floating-point arithmetic: `echo "0.5 * $base * $height" | bc -l`.
- `printf "%.2f"` rounds the circle's area to two decimal places.
- The `*)` pattern handles invalid menu choices.

Q2. Create a shell script that finds and lists all empty files in a /root directory.

An **empty file** is a regular file of size 0 bytes. The `find` command can locate these with `-type f -empty`. The script defaults to `/root` but accepts any directory as an argument (useful for testing). It must be run as root to read `/root`.

Script: q2_empty.sh

```
#!/bin/bash
# Q2: Find and list all empty files in the /root directory
# Usage: ./q2_empty.sh [directory]   (default: /root)

DIR=${1:-/root}
```

```
if [ ! -d "$DIR" ]; then
    echo "Error: $DIR is not a directory"
    exit 1
fi

echo "Empty files in $DIR:"
echo "----------------------------"
find "$DIR" -type f -empty > /tmp/empty_files.txt 2>/dev/null

if [ -s /tmp/empty_files.txt ]; then
    cat /tmp/empty_files.txt
    echo "----------------------------"
    echo "Total empty files: $(wc -l < /tmp/empty_files.txt)"
else
    echo "No empty files found."
fi
rm -f /tmp/empty_files.txt
```

Sample output (run as ./q2_empty.sh /tmp/td, a test directory)

```
Empty files in /tmp/td:
----------------------------
/tmp/td/empty1
/tmp/td/sub/empty2
----------------------------
Total empty files: 2
```

Explanation

- `DIR=${1:-/root}` takes the first argument `$1`, or `/root` if none is given.
- `[ ! -d "$DIR" ]` checks that the directory exists; `exit 1` stops with an error code otherwise.
- `find "$DIR" -type f -empty` searches the directory **and its sub-directories**; `2>/dev/null` hides permission errors.
- `[ -s file ]` is true when the temporary result file is not empty, so the list and the count (`wc -l`) are displayed; otherwise a 'No empty files' message appears.
- Only the top level of /root: add `-maxdepth 1` to find. One-line alternative: `find /root -type f -empty`. To also delete them: add `-delete` (use carefully).

Q3. Write a script to find whether entered number is even or odd.

A number is **even** if it is exactly divisible by 2 (remainder 0 when divided by 2); otherwise it is **odd**. The modulus operator `%` gives the remainder.

Script: q3_evenodd.sh

```
#!/bin/bash
# Q3: Check whether a number is even or odd

read -p "Enter a number: " num

# validate: optional minus sign followed by digits only
if ! [[ $num =~ ^-?[0-9]+$ ]]; then
    echo "Error: '$num' is not a valid integer."
    exit 1
fi

if [ $((num % 2)) -eq 0 ]; then
    echo "$num is an EVEN number."
else
```

```
    echo "$num is an ODD number."
fi
```

Sample output

```
$ ./q3_evenodd.sh
Enter a number: 24
24 is an EVEN number.

$ ./q3_evenodd.sh
Enter a number: 7
7 is an ODD number.

$ ./q3_evenodd.sh
Enter a number: abc
Error: 'abc' is not a valid integer.
```

Explanation

- `$((num % 2))` computes the remainder; `-eq 0` tests for equality with zero.
- The regular expression `^-?[0-9]+$` accepts only whole numbers (with an optional minus sign), so text input does not cause errors. Negative numbers work, e.g. -4 is even.
- Short alternative without validation: `[ $((num%2)) -eq 0 ] && echo even || echo odd`.

Q4. Write a program using case statement to perform Character Type Checker (Vowel, Consonant, Digit, Special Character).

The script accepts one character and uses `case` with **pattern matching (globbing)**. The order of patterns matters: vowels are checked first, so the later `[a-zA-Z]` pattern only matches consonants.

Script: q4_chartype.sh

```
#!/bin/bash
# Q4: Character type checker using case

read -p "Enter a single character: " ch

# accept exactly one character
if [ ${#ch} -ne 1 ]; then
    echo "Please enter exactly ONE character."
    exit 1
fi

case $ch in
  [aeiouAEIOU])
    echo "'$ch' is a Vowel."
    ;;
  [a-zA-Z])
    echo "'$ch' is a Consonant."
    ;;
  [0-9])
    echo "'$ch' is a Digit."
    ;;
  *)
    echo "'$ch' is a Special Character."
    ;;
esac
```

Sample output

```
Enter a single character: e      ->  'e' is a Vowel.
Enter a single character: b      ->  'b' is a Consonant.
Enter a single character: 5      ->  '5' is a Digit.
Enter a single character: @      ->  '@' is a Special Character.
Enter a single character: ab     ->  Please enter exactly ONE character.
```

Explanation

- `${#ch}` gives the length of the variable; the script rejects input that is not exactly one character.
- `[aeiouAEIOU]` matches any one vowel (upper or lower case); `[a-zA-Z]` matches any letter; `[0-9]` matches a digit.
- `*)` is the default case, so anything else (such as @, #, $, space) is a special character.
- Each case block ends with `;;`, and the statement ends with `esac`.

Q5. Write a program using case statement to perform Linux Command Information (menu: pwd, date, whoami, hostname).

Script: q5_cmdinfo.sh

```
#!/bin/bash
# Q5: Linux command information menu using case

echo "===== LINUX COMMAND INFORMATION ====="
echo "1. pwd"
echo "2. date"
echo "3. whoami"
echo "4. hostname"
read -p "Enter your choice (1-4): " choice

case $choice in
  1) echo "Current directory: $(pwd)" ;;
  2) echo "Current date and time: $(date)" ;;
  3) echo "Logged-in user: $(whoami)" ;;
  4) echo "Hostname: $(hostname)" ;;
  *) echo "Invalid choice! Enter a number between 1 and 4." ;;
esac
```

Sample output

```
$ ./q5_cmdinfo.sh
===== LINUX COMMAND INFORMATION =====
1. pwd
2. date
3. whoami
4. hostname
Enter your choice (1-4): 1
Current directory: /home/claude/sh

Choice 2 -> Current date and time: Tue Oct 13 20:02:42 UTC 2026
Choice 3 -> Logged-in user: root
Choice 4 -> Hostname: vm
Choice 7 -> Invalid choice! Enter a number between 1 and 4.
```

What each command shows

Command	Output
pwd	Print working directory: the full path of the current directory
date	Current system date and time

Command	Output
whoami	User name of the person running the script
hostname	Name of the computer

Explanation

- `$(command)` runs the command and inserts its output into the message.
- Each case option is written on one line, ending with `;;`.
- `*)` catches any number or text not in the menu.

Q6. Write a program using case statement for Linux User Management (Create, Delete, Lock, Unlock, List Users) - display the corresponding commands.

As the question asks to **display the commands**, the script prints the proper command for each menu option using the user name entered. This makes it safe to run (nothing is changed on the system). To really execute an action, replace the `echo` line with the command itself, for example `useradd "$user"`, and run the script as root.

Script: q6_usermgmt.sh

```
#!/bin/bash
# Q6: Linux user management menu - displays the corresponding commands

echo "===== LINUX USER MANAGEMENT ====="
echo "1. Create User"
echo "2. Delete User"
echo "3. Lock User"
echo "4. Unlock User"
echo "5. List Users"
read -p "Enter your choice (1-5): " choice

case $choice in
  1)
    read -p "Enter username to create: " user
    echo "Command to create user:"
    echo "  useradd $user"
    echo "  passwd $user        # set the password"
    ;;
  2)
    read -p "Enter username to delete: " user
    echo "Command to delete user:"
    echo "  userdel $user       # keep home directory"
    echo "  userdel -r $user    # also remove home directory and mail"
    ;;
  3)
    read -p "Enter username to lock: " user
    echo "Command to lock user:"
    echo "  passwd -l $user"
    echo "  (or)  usermod -L $user"
    ;;
  4)
    read -p "Enter username to unlock: " user
    echo "Command to unlock user:"
    echo "  passwd -u $user"
    echo "  (or)  usermod -U $user"
    ;;
  5)
    echo "Commands to list users:"
    echo "  cut -d: -f1 /etc/passwd          # all users"
    echo "  awk -F: '\$3>=1000 {print \$1}' /etc/passwd   # normal users"
    echo "  getent passwd"
```

```
    ;;
  *)
    echo "Invalid choice! Enter a number between 1 and 5."
    ;;
esac
```

Sample output

```
$ ./q6_usermgmt.sh
===== LINUX USER MANAGEMENT =====
1. Create User
2. Delete User
3. Lock User
4. Unlock User
5. List Users
Enter your choice (1-5): 1
Enter username to create: ravi
Command to create user:
  useradd ravi
  passwd ravi        # set the password

Enter your choice (1-5): 3
Enter username to lock: ravi
Command to lock user:
  passwd -l ravi
  (or)  usermod -L ravi

Enter your choice (1-5): 5
Commands to list users:
  cut -d: -f1 /etc/passwd          # all users
  awk -F: '$3>=1000 {print $1}' /etc/passwd   # normal users
  getent passwd
```

Commands used

Task	Command	Notes
Create user	useradd username; passwd username	Creates the account and home directory; passwd sets the password
Delete user	userdel username / userdel -r username	-r also deletes the home directory and mail spool
Lock user	passwd -l username / usermod -L username	Puts '!' before the password hash in /etc/shadow, so password login is disabled
Unlock user	passwd -u username / usermod -U username	Removes the lock
List users	cut -d: -f1 /etc/passwd	User names are the first field of /etc/passwd; UID 1000 and above are normal users

Explanation

- `read -p` asks for the user name, which is stored in `$user` and shown inside each command.
- In the list option, `\$3` and `\$1` are escaped inside `echo "..."` so that the shell prints the awk fields literally instead of replacing them.
- Locking only blocks password authentication; SSH key login may still work, so also use `chage -E 0 username` to expire the account fully.
- Check a lock with `passwd -S username` or `grep username /etc/shadow`.